

Deployment & DevOps

Original documentation: https://docs.google.com/document/d/1yJLejGVrdJQ4Gq0f72P750_r9pzv-XMS-WSrUH_jQEI/edit?usp=sharing

Overview

The platform runs as three services via Docker Compose: API (Node/Express), PostgreSQL, and Redis. This provides a reproducible local environment and a production-like deployment model.

Diagram

Deployment diagram is stored in docs/assets/diagrams/deployment_architecture_diagram.jpg.

How to Run (Local)

- * Create a .env file (or set environment variables) for the backend.
- * Run `docker compose up --build`.
- * Verify health: `GET /api/health` returns 204.

Environment Variables

Variable	Default	Required	Description	Example
PORT	4000	Yes	The port on which the Express API server...	4000
NODE_ENV	development	No	Runtime environment mode. See the prod...	development
DATABASE_URL	-	Yes	Full PostgreSQL connection string. Use...	postgres://dev_user:password@localh...
POSTGRES_USER	-	Yes	PostgreSQL username. Used by the db c...	postgres
POSTGRES_PASSWORD	-	Yes	PostgreSQL password. Used by the db c...	password
POSTGRES_DB	-	Yes	PostgreSQL database name. Used by the...	dev_db
REDIS_HOST	localhost	Yes	Hostname or IP of the Redis server. S...	localhost
REDIS_PORT	6379	Yes	Port on which Redis is listening.	6379
SENTRY_DSN	-	No	Sentry project DSN for error tracking...	https://...
CORS_ORIGIN	*	No	Allowed origin(s) for CORS. See the...	http://localhost:5173
API_URL	-	No	Base URL of the backend API. Used by...	http://localhost:4000/api
CLIENT_URL	-	No	Base URL of the frontend application...	http://localhost:5173
LOG_LEVEL	info	No	Minimal log level for the application...	debug

Notes

- * Health checks are configured for API/DB/Redis in `docker-compose.yml`.
- * Persistent data is stored in Docker volumes for PostgreSQL and Redis.